
MOBIUS

R O G U E L I K E

Rapport de soutenance — Projet Jeu-Vidéo

EPITA — Promo 2030 — Soutenance du 27 mars 2026

M3G_STUDIO

Studio de développement indépendant

Georges Boyer	Chef de groupe — Développement principal
Madeleine Diop	Environnement visuel & graphismes
Emile Levast	Lore, narration & site web
Maël Bosse-Platière	Multijoueur en ligne

27 mars 2026

Contents

1	Introduction	2
1.1	Présentation du projet	2
1.2	Présentation du groupe	2
1.3	But du projet	3
1.4	Présentation de l'entreprise fictive	4
1.5	État de l'art & inspirations	4
1.6	Répartition des tâches	5
1.7	Ressenti global du projet	5
2	Avancement du projet	6
2.1	Scénario et univers	6
2.2	Mécaniques de jeu	6
2.3	Fonctionnalités	7
2.4	Multijoueur en ligne	8
2.5	Graphismes et environnement visuel	9
2.6	Architecture du code	10
3	Bilan de l'avancement	12
3.1	Ce qui a été réalisé	12
3.2	Difficultés et ajustements	12
3.3	Écarts prévisionnel / réel	13
4	Communication	14
4.1	Site web M3G_STUDIO	14
4.2	Logo et identité visuelle	14
4.3	Manuel d'utilisation	14
5	Conclusion	15
A	Annexes illustrées	16
A.1	Menus	16
A.2	Gameplay — Époques	17
A.3	Multijoueur	19
A.4	Communication & planification	19
B	Liens et ressources en ligne	19

1 Introduction

Ce rapport de soutenance présente l'avancement du projet **MOBIUS Roguelike**, réalisé dans le cadre du projet *Jeu-Vidéo* de première année à l'EPITA (promo 2030). Il retrace les choix techniques et créatifs effectués par l'équipe M3G_STUDIO, les fonctionnalités implémentées, ainsi que le bilan organisationnel du groupe.

1.1 Présentation du projet

MOBIUS est un jeu de type *roguelike* en vue du dessus (*top-down shooter*) développé entièrement en Python avec la bibliothèque Pygame. Le principe central est un voyage à travers six époques historiques : le joueur affronte des vagues d'ennemis propres à chaque ère, récupère des armes dans des coffres, et tente de survivre jusqu'à l'ère Futuriste.

Le jeu propose deux modes : un mode **solo** et un mode **multijoueur en ligne** (co-op 2 joueurs via UDP), chacun supportant la sélection d'une classe de personnage parmi cinq disponibles.

Fiche technique

Moteur / Langage	Python 3.11+ / Pygame 2.x
Résolution	Plein écran adaptatif (détection dynamique)
Réseau	UDP maison, port 55 555, architecture client-serveur
Époques	6 : Préhistoire → Grèce → Edo → Moderne → WW2 → Futur
Classes	5 : Tank, Berserker, Vampire, Ninja, Archer
Armes	Distanciel, mêlée, AOE selon l'époque

1.2 Présentation du groupe

Notre groupe M3G_STUDIO est composé de quatre étudiants en première année à l'EPITA (promo 2030), réunis autour d'une passion commune pour les jeux vidéo et la programmation.

Georges Boyer — Chef de groupe, développement principal

Passionné de programmation, Georges a naturellement pris en charge l'architecture générale du projet. Il a conçu le moteur de jeu (`base_room.py`), l'orchestrateur principal (`main.py`), le système de mécaniques (`mechanics.py`) et les constantes globales (`constants.py`). En tant que chef de groupe, il a également coordonné les réunions d'avancement et l'intégration des différents modules.

Madeleine Diop — Environnement visuel & graphismes

Madeleine a pris en charge tout le rendu visuel du jeu : création des fonds thématiques par époque, effets de particules (pétales de cerisier dans l'ère Edo, pluie WW2, fumée napoléonienne, scan holographique futuriste), et cohérence graphique globale. Elle a développé le module `graphics.py` comprenant le `SpriteCache`, le `BackgroundRenderer` et les `ScreenEffects`.

“Rendre chaque époque unique visuellement sans alourdir les performances a été mon défi principal. Les effets de particules différenciés par époque ont vraiment enrichi l’immersion.”

Emile Levast — Lore, narration & site web

Emile s’est chargé de l’univers narratif du jeu et de la communication externe. Il a défini le *lore* de MOBIUS (l’histoire du voyageur temporel, la logique des six époques), rédigé les textes du jeu, et développé le site web de M3G_STUDIO.

Maël Bosse-Platière — Multijoueur en ligne

Maël a développé le module réseau complet du jeu (`network.py`). Il a implémenté une architecture client-serveur UDP maison : le `GameServer` joue le rôle de serveur autoritaire, le `GameClient` envoie ses inputs et reçoit l’état du jeu pour l’afficher via le `ClientRenderer`.

1.3 But du projet

Origine

M3G_STUDIO est né de la volonté de créer un jeu original qui dépasse le simple exercice scolaire. L’idée centrale — traverser les grandes époques de l’humanité en affrontant des ennemis historiques — est née d’une réflexion sur ce qui rend un roguelike rejouable : la variation. En changeant radicalement d’univers visuel, d’armes et d’ennemis à chaque époque, chaque *run* offre une expérience différente.

Nature du jeu

MOBIUS est un *top-down shooter roguelike* : le joueur évolue dans des arènes en vue aérienne, élimine des vagues d’ennemis de difficulté croissante, et récupère des améliorations (*power-ups*) et armes dans des coffres. La mécanique de progression par époques — chacune avec ses propres ennemis, armes et ambiance visuelle — est le fil conducteur du jeu.

Objectif

Livrer un jeu jouable, complet et soigné : menus animés, sélection de classe, six époques jouables, mode solo et multijoueur en ligne, et un rendu visuel cohérent avec l'identité de M3G_STUDIO.

1.4 Présentation de l'entreprise fictive

M3G_STUDIO a été fondé pour ce projet par les quatre membres de l'équipe. Le nom reprend les initiales des fondateurs et affirme une identité collective. Notre studio se spécialise dans les jeux indépendants à fort caractère narratif et visuel, en particulier les roguelikes et les jeux d'action à progression.

Notre premier titre, **MOBIUS Roguelike**, illustre notre philosophie : un gameplay accessible mais profond, une direction artistique soignée, et une rejouabilité portée par la variation des univers.

1.5 État de l'art & inspirations

Le genre roguelike top-down est largement représenté dans l'industrie indépendante. Parmi les titres de référence :

- **Enter the Gungeon** (Dodge Roll, 2016) : référence du top-down shooter roguelike avec progression par armes.
- **Hades** (Supergiant Games, 2020) : roguelike narratif avec runs courts et progression méta.
- **Nuclear Throne** (Vlambeer, 2015) : roguelike action intense avec personnages aux capacités distinctes.
- **Vampire Survivors** (poncle, 2022) : popularisation du *bullet heaven* avec progression automatique.
- **Call of Duty Zombies** (Treyarch, 2008) : inspiration pour le système de vagues d'ennemis croissantes.

MOBIUS se différencie par son système d'époques historiques qui transforme radicalement l'ambiance et les mécaniques à chaque niveau, et par son mode co-op en ligne intégré. **Enter the Gungeon** a directement inspiré notre système d'armes variées et la mécanique de coffres. La sélection de classe s'inspire de **Hades**. La progression en vagues s'inspire de **Vampire Survivors** et de **Call of Duty Zombies**.

1.6 Répartition des tâches

Table 1: Répartition des tâches par membre

Membre	Périmètre	État
Georges Boyer	Architecture (base_room, main, mechanics, constants), IA ennemis, système d'armes, classes joueur, coffres, power-ups	En cours
Madeleine Diop	Graphismes (graphics.py), effets par époque, backgrounds, HUD, sprites	En cours
Emile Levast	Lore des époques, textes in-game, site web M3G_STUDIO	Terminé
Maël Bosse-Platière	Module réseau UDP (network.py), synchronisation client-serveur, gestion déconnexion	En cours

1.7 Ressenti global du projet

Le développement de MOBIUS a été une expérience intense et formatrice. Partir d'une idée et aboutir à un jeu fonctionnel avec menu, multijoueur, six univers distincts et un système de classes en quelques semaines nous a confrontés à des défis bien au-delà du simple cours de Python. Les difficultés ont été nombreuses — synchronisation réseau, gestion des ressources visuelles, cohérence des états du jeu — mais chaque obstacle résolu a renforcé notre maîtrise des outils et notre dynamique d'équipe.

2 Avancement du projet

2.1 Scénario et univers

Contexte narratif

Le joueur incarne un voyageur temporel coincé dans une boucle infinie — une *bande de Möbius* — qui le fait traverser six époques de l'histoire humaine. À chaque époque, des créatures hostiles ont envahi la ligne temporelle et menacent l'intégrité du continuum. Le joueur doit les éliminer pour passer à l'époque suivante, en s'adaptant aux armes et aux techniques de combat propres à chaque ère.

Progression par époques

Table 2: Époques, thèmes et multiplicateurs de difficulté

Ordre	Époque	Difficulté	Ambiance
1	Préhistoire	×1.0	Âge de pierre, cavernes, feux de camp
2	Grèce Antique	×1.3	Temples, Olympe, soleil méditerranéen
3	Période Edo	×1.6	Japon médiéval, cerisiers, nuit étoilée
4	Ère Moderne	×2.0	Napoléon, champs de bataille, fumée de canon
5	Guerre Mondiale	×2.5	WW2, jungle, bunker, pluie
6	Ère Futuriste	×3.0	Station spatiale, néons, hologrammes

Transitions entre époques

Les transitions sont gérées par la classe `EpochTransition` dans `main.py` : un fondu noir progressif accompagné du nom de l'époque en grand texte. La progression des stats du joueur (PV, stamina, kills, coins) est conservée entre les époques.

2.2 Mécaniques de jeu

Déplacements du joueur

Le joueur se déplace au clavier (ZQSD) et vise à la souris. Un système de **dash** (touche `ESPACE`) permet une esquive rapide avec cooldown (`DASH_COOLDOWN = 30` frames) et coût en stamina (`DASH_STAMINA_COST = 15`). La vitesse de base est modulée par la classe choisie.

Système d'armes

Les armes sont définies dans `WEAPONS_DATA` et instanciées via la classe `Weapon`. Chaque arme possède :

- Un type (`ranged` ou `melee`)
- Des dégâts (`damage`), un cooldown (`cooldown_max`) et un coût en stamina
- Pour les armes à distance : vitesse de projectile (`projectile_speed`)
- Pour la mêlée : portée d'arc d'attaque (`melee_range`)

Les projectiles joueur (`Bullet`) héritent du sprite de l'arme, rotationné selon la direction de tir. Les attaques de mêlée (`MeleeAttack`) génèrent une ellipse d'arc positionnée dans la direction visée.

Classes de personnage

Table 3: Classes disponibles et leurs caractéristiques

Classe	Stats	Compétence spéciale
Tank	150 PV, Vitesse -30%	Bouclier : -50% dégâts reçus
Berserker	80 PV, Vitesse $+30\%$	Rage : $\times 2$ dégâts pendant 5s
Vampire	Stats normales	$+10$ PV par kill (10s)
Ninja	PV réduits, Vitesse max	Invisibilité brève
Archer	Stats normales	Flèche multiple à distance

Power-ups et coffres

Les **power-ups** (classe `PowerUp`) apparaissent aléatoirement et offrent quatre types de bonus : *Damage*, *Speed*, *Health*, *Stamina*. Ils pulsent visuellement et disparaissent après 600 frames s'ils ne sont pas ramassés.

Les **coffres** (classe `Chest`) contiennent une arme spécifique à l'époque. Le joueur les ouvre en s'approchant (touche E), déclenchant une animation d'ouverture et l'ajout de l'arme à son inventaire.

2.3 Fonctionnalités

Menu principal & navigation

Le menu principal (`MainMenuRenderer`) affiche le logo MOBIUS avec une animation flottante, des particules de fond et deux boutons : *Jouer* et *Quitter*. L'écran de mode

(`ModeSelectRenderer`) présente deux grandes cartes cliquables — **Solo** et **En ligne** — avec effets de survol animés. Un slider de volume persistant (`VolumeSlider`) est présent sur tous les écrans de menus.

Figure 1: ref annexe: Écran titre — Menu principal MOBIUS

Sélection de classe

L'écran `MenuRenderer` affiche cinq cartes de classe avec dégradé de couleur, icône, description et compétence spéciale. Un effet de surbrillance et d'élévation indique la carte survolée. La sélection se fait au clic ou via les touches 1–5.

HUD (affichage tête haute)

Le HUD affiche en temps réel : barre de PV, barre de stamina, arme courante avec cooldown visuel, nom de l'époque, numéro de vague, compteur de kills et de pièces collectées, et indicateur de compétence spéciale.

Figure 2: ref annexe: HUD pendant une partie

2.4 Multijoueur en ligne

Architecture réseau

Le module `network.py` implémente une architecture **client-serveur autoritaire sur UDP** (port 55555 par défaut). Le **HOST** est le serveur : il simule l'intégralité du jeu et envoie l'état complet au client à chaque frame. Le **CLIENT** n'effectue aucune simulation locale : il envoie ses inputs (déplacement, tir, dash...) et reçoit l'état du jeu via le `ClientRenderer`.

Flux de connexion

1. Le HOST lance le serveur UDP et attend le client (écran *En attente de P2...*).
2. Le CLIENT saisit l'IP du host (`IPInputRenderer`) et se connecte dans un thread séparé pour ne pas bloquer l'UI.
3. Une fois connectés, chaque joueur arrive sur l'écran de sélection de classe.
4. Le HOST envoie le signal `send_start` ; le jeu démarre simultanément.

Synchronisation

Éléments synchronisés à chaque frame :

- Position et direction du joueur P2

- Points de vie de P2
- État des ennemis (position, PV)
- Projectiles ennemis actifs
- Transitions d'époques et événements *game-over*

La gestion des timeouts (8 secondes) permet de détecter proprement une déconnexion.

Défis rencontrés

L'implémentation réseau a posé plusieurs problèmes techniques : la sérialisation efficace de l'état du jeu (nombreux sprites), la gestion de la latence variable en UDP, et la synchronisation des transitions d'époque entre les deux machines. La solution retenue — serveur autoritaire sans prédiction client — privilégie la simplicité et la cohérence.

2.5 Graphismes et environnement visuel

Décors par époque

Chaque époque dispose d'une classe héritant de `BaseRoom` et implémentant `draw_epoch_decoration()` :

Table 4: Effets visuels par époque

Époque	Couleur thématique	Effet de décoration
Préhistoire	Brun (139, 69, 19)	Flammes animées (optionnelles)
Grèce Antique	Or grec (200, 180, 80)	Soleil animé (optionnel)
Période Edo	Rouge (180, 30, 30)	Pétales de cerisier flottants
Ère Moderne	Bleu (0, 80, 160)	Volutes de fumée de canon
Guerre Mondiale	Vert (60, 80, 40)	Pluie battante
Ère Futuriste	Cyan (0, 220, 255)	Scan holographique + chiffres 0/1

Système de sprites et cache

Le module `graphics.py` centralise le chargement des assets via la classe `SpriteCache` (singleton) qui évite les rechargements redondants. Le `BackgroundRenderer` génère les fonds thématiques par époque. Les `ScreenEffects` gèrent les flashes d'impact, les fades et les effets de transition.

(a) ref annexe: Ère Edo

(b) ref annexe: Ère Futuriste

Figure 3: ref annexe: Exemples de décorations d'époque

2.6 Architecture du code

Structure modulaire

```

mobius/
|-- main.py          # Orchestrateur, etats, menus, boucle principale
|-- core/
|   |-- constants.py # Couleurs, donnees armes, skills, epochs
|   |-- graphics.py  # SpriteCache, BackgroundRenderer, ScreenEffects
|   |-- mechanics.py  # Weapon, Bullet, MeleeAttack, PowerUp, Chest
|   |-- base_room.py  # Classe mere de toutes les salles
|   '-- network.py    # GameServer, GameClient, ClientRenderer
'-- epoques/
    |-- prehistoire.py # Salle ere Prehistorique
    |-- grece.py       # Salle Grece Antique
    |-- edo.py         # Salle Periode Edo
    |-- moderne.py     # Salle Ere Moderne
    |-- contemporain.py # Salle Guerre Mondiale
    '-- futuristique.py # Salle Ere Futuriste

```

Machine à états

La classe `Game` gère l'état global du jeu via des constantes entières :

Table 5: États du jeu et transitions

État	Description	Transition vers
MAIN_MENU	Écran titre	MODE_SELECT
MODE_SELECT	Solo ou En ligne	CHARACTER_SELECT / ONLINE_MENU
ONLINE_MENU	Héberger / Rejoindre	WAITING_CLIENT / IP_INPUT
CHARACTER_SELECT	Choix de classe	PLAYING
PLAYING	Jeu en cours	GAME_OVER
GAME_OVER	Fin de partie	MAIN_MENU / PLAYING

Extrait de code représentatif

```

1 def change_epoch(self, next_epoch: str | None):
2     if next_epoch and next_epoch in self.rooms:
3         p = self.current_room.player

```

```
4         stats = {"skill": p.skill, "kills": p.kills,
5                  "coins": p.coins, "health": p.health,
6                  "max_health": p.max_health}
7     def _do():
8         self.current_epoch = next_epoch
9         self.current_room = self.rooms[next_epoch]
10        self.current_room.start(
11            stats["skill"], mode=self.net_mode,
12            player_stats=stats, server=self.server)
13        if self.server:
14            self.server.send_start(stats["skill"])
15        self.transition.start(next_epoch, _do)
16    else:
17        self.game_state = GAME_OVER
```

Listing 1: Méthode `change_epoch()` dans `main.py` — transition d'époque avec persistance des stats

3 Bilan de l'avancement

3.1 Ce qui a été réalisé

L'ensemble des fonctionnalités prévues en priorité haute a été livré :

- ✓ Six époques jouables avec décors distincts, ennemis et armes thématiques
- ✓ Cinq classes de personnage avec compétences spéciales
- ✓ Système d'armes complet (distanciel, mêlée, projectiles ennemis)
- ✓ Coffres et power-ups avec animations
- ✓ Menus animés complets (titre, mode, classe, game over)
- ✓ Transitions d'époques avec fondu et texte animé
- ✓ Mode solo pleinement fonctionnel
- ✓ Module réseau UDP avec architecture client-serveur
- ✓ HUD complet (PV, stamina, arme, vague, kills, coins)
- ✓ Logo et identité visuelle M3G_STUDIO
- ✓ Site web M3G_STUDIO (Emile Levast)

3.2 Difficultés et ajustements

- **Flammes préhistoriques et soleil grec** : ces effets ont été désactivés pour raisons de performance. Ils restent commentés dans le code et sont réactivables facilement.
- **Synchronisation réseau complète** : le rendu client est fonctionnel mais certains effets de particules ne sont pas transmis. La synchronisation des sons reste partielle.
- **Mode LAN vs Internet** : le multijoueur fonctionne de façon fiable en réseau local. Sur Internet, l'ouverture du port UDP 55555 reste nécessaire côté hôte.

3.3 Écarts prévisionnel / réel

Table 6: Comparaison prévisionnel vs réalisé

Fonctionnalité	Prévu	Réalisé
6 époques jouables	Oui	Oui
5 classes joueur	Oui	Oui
Mode solo	Oui	Oui
Mode multijoueur LAN	Oui	Oui
Mode multijoueur Internet	Oui	Partiel
Effets décor toutes époques	Oui	4/6 actifs
Site web M3G_STUDIO	Oui	Oui
Sons / Musique	Bonus	Partiel

4 Communication

4.1 Site web M3G_STUDIO

Le site web de M3G_STUDIO a été réalisé par Emile Levast. Il présente le studio, le jeu MOBIUS, les membres de l'équipe, et propose un lien de téléchargement du jeu. Le site reflète l'identité visuelle de MOBIUS et inclut des pages dédiées au lore des six époques et aux contrôles du jeu.

Figure 4: ref annexe: Site web M3G_STUDIO

4.2 Logo et identité visuelle

Le logo MOBIUS a été conçu en pixel-art avec un dégradé cyan vers bleu profond, agrémenté d'un effet de foudre rose en bas. Ce logo est utilisé de façon cohérente sur l'écran titre, dans tous les menus, sur le site web et dans les documents du projet.

4.3 Manuel d'utilisation

Les contrôles du jeu sont affichés directement dans l'interface de sélection de mode, garantissant que le joueur est informé avant de commencer.

Table 7: Contrôles du jeu

Touche	Action
ZQSD	Déplacement
Clic gauche	Tirer / Attaquer
ESPACE	Dash (esquive)
F	Compétence spéciale
E	Ouvrir un coffre
1 / 2	Changer d'arme
ESC	Menu pause / Retour

5 Conclusion

Le développement de MOBIUS Roguelike a représenté pour M3G_STUDIO bien plus qu'un exercice académique. En partant de zéro pour concevoir un jeu complet — architecture modulaire, graphismes thématiques, système de classes, et multijoueur en ligne — nous avons collectivement franchi un cap significatif dans notre maîtrise de Python et de la conception logicielle.

Le résultat est un jeu jouable, cohérent visuellement et narrativement, avec une rejouabilité portée par les six univers distincts et les cinq classes. Le module réseau UDP, développé sans framework, constitue techniquement la partie la plus ambitieuse du projet.

Ce que ce projet nous a appris dépasse le code : planification réaliste, communication régulière, gestion des imprévus, et fierté collective d'un livrable abouti. Ces compétences nous accompagneront bien au-delà de la promo 2030.

M3G_STUDIO — MOBIUS Roguelike — EPITA 2026

“Every loop ends. Until it doesn't.”

A Annexes illustrées

Cette section est réservée aux captures d'écran et illustrations du jeu. Pour intégrer une image, remplacer chaque `\fbbox` par :

```
\includegraphics[width=0.8\textwidth]{screenshots/nom_image.png}
```

A.1 Menus

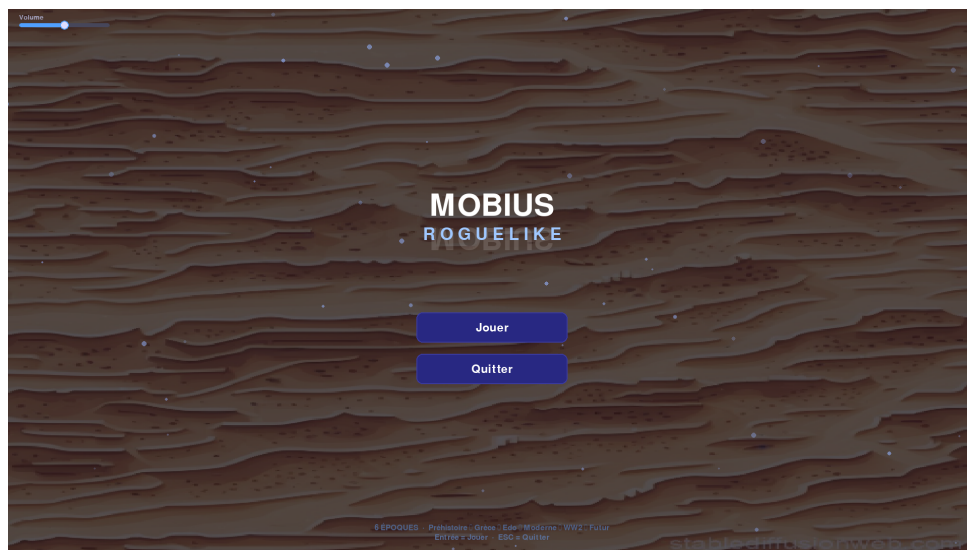


Figure 5: Annexe A1 — Menu principal



Figure 6: Annexe A2 — Sélection de mode

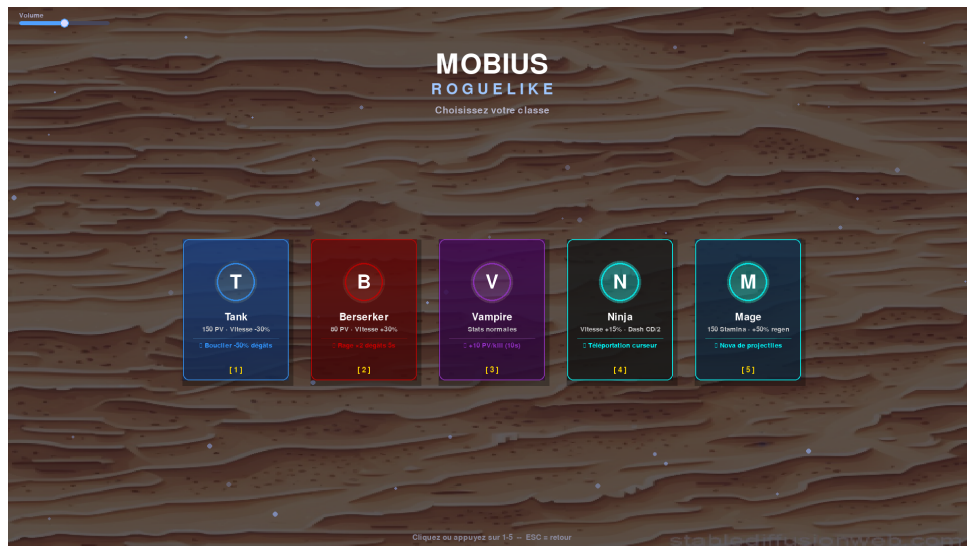


Figure 7: Annexe A3 — Sélection de classe

A.2 Gameplay — Époques



Figure 8: Annexe B1 — Ère Préhistorique

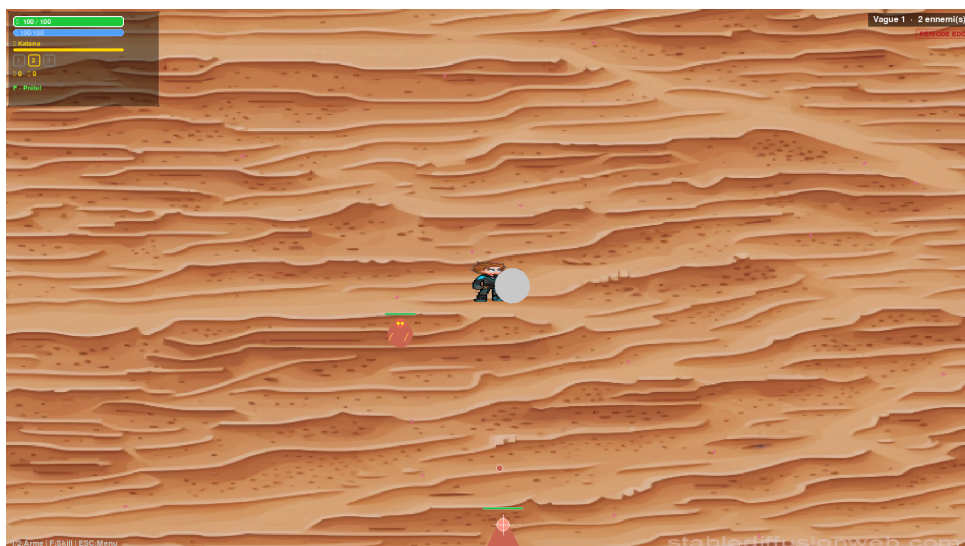


Figure 9: Annexe B2 — Période Edo



Figure 10: Annexe B3 — Ère Futuriste

A.3 Multijoueur



Figure 11: Annexe C1 — Attente multijoueur (host)

A.4 Communication & planification

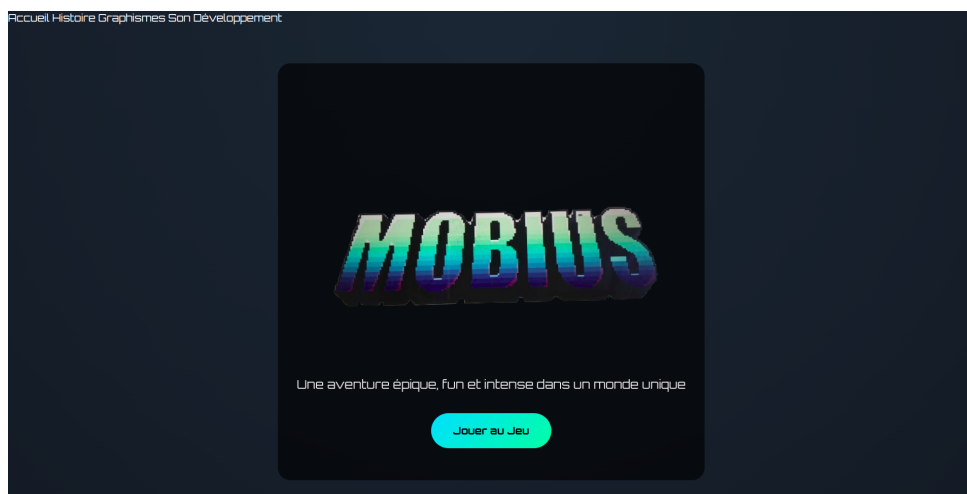


Figure 12: Annexe D1 — Site web M3G_STUDIO

B Liens et ressources en ligne

Cette annexe regroupe les liens vers les ressources accessibles en ligne pour compléter la présentation du projet.

Dépôt de code source

GitHub M3G_STUDIO https://github.com/Epigold/Mobius_EPITA/

Note : les URLs ci-dessus sont les adresses prévues au moment de la soutenance. En cas d'indisponibilité, contacter directement l'équipe M3G_STUDIO.